MEDSYS SPECIFICATION SERIES

MEDSYS Frontend Client Specification

Client overview, scope, user flows, requirements baseline, and implementation position for the MEDSYS frontend.

Client Review Draft | Frontend 0.1.0 | March 12, 2026

AUDIENCE
Clients, sponsors, business stakeholders, delivery management

DOCUMENT VERSION
1.0

SYSTEM VERSION
Frontend 0.1.0

STATUS
Client Review Draft

DOCUMENT INTENT
This specification provides the full client-facing frontend baseline without removing sections from the original documentation structure.

1. Executive Summary

Medsys is a web-based healthcare operations platform designed for internal clinic use. The current product supports role-based workflows for owners, doctors, and assistants across patient management, appointments, encounters, prescriptions, inventory, analytics, AI-assisted summaries, and patient-profile viewing.

The application is positioned as an operational clinical workspace rather than a public patient portal. It is built to give staff one controlled interface for day-to-day patient flow, consultation support, dispensing support, staff administration, and operational visibility.

2. Purpose

This document provides a client-facing view of the Medsys frontend application. It explains:

- what the system is
- why it exists
- what it currently does
- which user roles it supports
- the major UI flows
- the current functional and non-functional requirements baseline
- the current technology baseline

This document is intended to support product review, stakeholder communication, procurement discussion, implementation planning, and business sign-off conversations.

3. Aims And Objectives

The current Medsys implementation is intended to:

- centralize clinic operations in one browser-based workspace
- reduce fragmented staff workflows across patient intake, consultation, and dispensing
- provide clear role-based access to operational and clinical functions
- improve visibility of appointments, patient records, prescriptions, and inventory
- provide a maintainable path toward a production-grade healthcare platform

Current business objectives supported by the product include:

- faster patient handling from registration through consultation and dispensing
- clearer doctor access to patient context
- assistant support for intake and dispense queues
- owner visibility into staff and operational activity
- better operational oversight through analytics and AI summary views

4. Product Scope

In scope

- authentication and role-based access
- doctor workspace
- assistant workspace
- patient directory
- patient profile
- appointments
- encounters
- prescriptions and dispensing
- inventory
- analytics
- AI insights
- owner workspace

Out of scope

- patient self-service portal
- billing and claims workflows
- laboratory system integration
- imaging integration
- external pharmacy integration
- formal compliance certification
- advanced enterprise reporting

5. Product Overview

ROLE	CURRENT RESPONSIBILITIES
Owner	staff oversight, access visibility, high-level operational administration
Doctor	waiting queue review, patient context review, encounter submission, diagnosis, tests, and prescription entry
Assistant	patient intake, dispense queue handling, workflow coordination

The application uses a protected shell experience for authenticated users, with route and action visibility controlled by shared authorization rules.

6. Technology Stack

Current frontend technology baseline:

- Next.js 16
- React 19
- TypeScript 5
- Tailwind CSS 4
- TanStack Query
- Vitest and Testing Library

Current architecture baseline:

- browser calls same-origin `/api/...` routes
- frontend BFF routes call backend `/v1/...` services server-side
- signed application session cookies are used for app identity
- backend access and refresh tokens are stored in secure `httpOnly` cookies

This architecture keeps backend tokens out of browser JavaScript and provides a controlled integration layer between the UI and backend services.

7. User Roles And Permission Model

The current application supports three operational roles:

- `Owner`
- `Doctor`
- `Assistant`

Current permission behavior:

- route access is role-aware
- navigation visibility is permission-aware
- selected UI actions are aligned to backend policy
- internal API routes enforce shared permissions on the frontend BFF layer

Current important policy example:

- appointment creation is currently allowed for `Owner` and `Assistant`
- appointment creation is not currently allowed for `Doctor`

8. Core UI Flows

8.1 Login And Session Flow

1. User opens the login page.
2. Credentials are submitted to the frontend authentication route.
3. Frontend authenticates against the backend.
4. Signed app session and secure backend auth cookies are created.
5. User is redirected to the correct workspace based on role.

8.2 Doctor Workflow

1. Doctor enters the doctor workspace.
2. Waiting appointments are loaded.
3. Doctor selects a patient from the queue.
4. Patient context, vitals, and allergies are loaded when available.
5. Doctor records notes, diagnoses, tests, prescriptions, and next-visit information.
6. Encounter is submitted to the backend.

8.3 Assistant Workflow

1. Assistant enters the assistant workspace.
2. Pending dispense queue and supporting workflow data are loaded.
3. Assistant may register a new patient through intake.
4. Assistant selects a pending prescription.
5. Prescription details are displayed.
6. Assistant completes dispensing using available inventory.

8.4 Patient Directory And Profile Flow

1. User opens the patient directory.
2. Patient list is loaded.
3. User searches or selects a patient.
4. Patient profile view loads summary, family, allergies, conditions, vitals, and timeline feeds.
5. The screen shows warnings if some detail feeds are partially unavailable.

8.5 Owner Workflow

1. Owner opens the owner workspace.
2. Backend-backed staff list and operational visibility data are loaded.

3. Owner reviews staff access information.
4. Owner creates staff accounts through the users API.

8.6 Analytics And AI Flow

1. Analytics workspace loads operational overview and supporting feeds.
2. AI workspace derives summary insights from analytics, appointments, patients, and audit activity.
3. Both screens provide loading, error, empty, and retry states.

8.7 Inventory Flow

1. Inventory list loads.
2. User selects an item.
3. Item details and movement history are displayed.
4. User creates items or inventory movements through backend-backed routes.

9. Functional Requirements Baseline

The current functional requirements baseline includes the following major capabilities.

9.1 Authentication And Access

- login
- logout
- authenticated session handling
- route protection
- role-based workspace routing
- backend-aware auth status checks

9.2 Doctor Functions

- waiting queue display
- patient context loading
- encounter notes
- diagnoses
- tests
- prescription entry
- next visit entry
- encounter submission

9.3 Assistant Functions

- intake form
- patient creation
- pending dispense queue
- prescription detail display
- dispense action completion

9.4 Patient Functions

- patient directory listing
- patient search
- patient profile launch
- patient history access

9.5 Owner Functions

- staff visibility
- backend-backed staff account creation
- permission preset presentation

9.6 Operations Functions

- appointments listing and creation
- encounters listing and submission
- inventory listing and movement creation
- analytics overview
- AI insight generation
- ICD-10 suggestion access

10. Non-Functional Requirements Baseline

The current non-functional requirements baseline includes:

10.1 Security

- secure cookie-based session model
- backend tokens not exposed to browser JavaScript
- same-origin BFF integration model
- protected shell routing
- permission-aware frontend route handling

10.2 Reliability And Error Handling

- backend availability gating
- dedicated unavailable and error pages
- explicit loading, empty, ready, and error states
- retry behavior on many data screens
- partial-data warnings where fallback rendering is used

10.3 Maintainability

- feature-organized section structure
- shared API client
- shared query/server-state layer
- shared authorization model
- centralized BFF integration pattern

10.4 Quality Control

- linting
- type checking
- unit and component testing
- route and hook coverage for core frontend behavior

10.5 Performance And UX

- responsive browser UI
- disabled actions during in-flight operations
- server-state caching and refetch handling through shared query hooks

11. Current Implementation Status

Current implementation position:

- frontend BE-020 integration closure is complete from the frontend scope
- active production browser flows use backend-backed `/api/...` BFF routes
- prototype persistence has been removed from the active frontend tree
- query-based server-state infrastructure is in place across the main read-heavy sections

Current product maturity statement:

- the application is structurally strong for continued development
- it is not yet a completed enterprise healthcare platform
- additional work is still required for deeper workflow hardening, compliance, and production operations

12. Known Limitations

Current known limitations include:

- no patient self-service or public portal
- no billing or claims module
- no full compliance sign-off package
- no complete audit-governance program at enterprise depth
- some workflows still require future backend policy refinement
- some advanced clinical safety and interoperability capabilities are not yet implemented

13. Version And Document Control

DOCUMENT VERSION	1.0
DOCUMENT DATE	March 12, 2026
SYSTEM VERSION REFERENCED	Frontend 0.1.0
STATUS	client review draft
RECOMMENDED USE	client presentation, project onboarding, scope clarification, product review, implementation discussion

14. Conclusion

Medsys currently provides a structured, role-based healthcare operations UI for internal clinic users. It already supports the major operational flows needed to manage patients, consultations, prescriptions, dispensing, inventory, analytics, and staff workflows within a controlled browser application.

The current baseline is suitable for stakeholder review, phased delivery planning, and ongoing implementation governance.

This HTML file is the styled source used to export the final PDF.